

Marine Debris Detection — The Full Story

Every Step Explained Simply

University Machine Learning Project • June 2026

From raw sonar images to a trained AI detector

What We Did	Best Score
Taught AI to name debris (Classification)	98.87% accuracy
Taught AI to find debris in a scene (Detection)	93.7% mAP50

Trained on Apple M4 Mac & Kaggle T4 GPU • PyTorch 2.12 • Python 3.11

Chapter 1: What Are We Trying to Do?

This is the "big picture" chapter. Before touching any code, you need to understand the goal.

The Problem

The ocean floor is full of garbage — old tires, cans, bottles, plastic pipes, chains, and more. This trash harms fish, corals, and the entire ocean ecosystem. But sending human divers to find and map all this garbage is dangerous, slow, and expensive.

The idea: What if we could send a robot submarine (called an AUV — Autonomous Underwater Vehicle) that has a camera-like sensor and an AI brain? The robot could scan the ocean floor by itself, find all the trash, and create a map of where it is.

Analogy — Robot Vacuum Cleaner

You know how a Roomba robot vacuum drives around your house and knows where the furniture is? Our project is like building a Roomba for the ocean — except instead of sucking up dust, it detects underwater garbage.

What Kind of Camera Did We Use?

A normal camera doesn't work well deep underwater — it's too dark. Instead, we used a **Forward-Looking Sonar (FLS)** sensor. Sonar works like a bat's echolocation: it sends out sound waves, and when those waves bounce off an object, they come back. The sensor draws a grayscale picture based on how strong the echo was.

- **Bright white areas** = solid objects (debris!) — strong echo
- **Dark black areas** = empty water or soft mud — weak echo

Analogy — Flashlight in the Dark

Imagine you're in a pitch-black room with only a flashlight. You point it around and see shapes. Sonar is like a sound flashlight — instead of light, it uses sound to 'see' in the dark water.

Our Two Tasks

We trained two different AI systems, each with a different job:

Task	What the AI does	Analogy
1. Classification	Given a cropped photo of ONE object, tell us what it is	Looking at a single flashcard and naming the animal on it
2. Detection	Given a full scene with MANY objects, draw boxes around each one and name them	Looking at a messy room photo and circling every piece of trash

Each task is harder than the previous one. Classification is the simplest (one answer for the whole image). Segmentation is the hardest (one answer for every single pixel in the image!).

Chapter 2: The Dataset — Our Training Material

An AI learns from examples, just like a student learns from textbooks. This chapter explains what data we used to train our AI.

Where Did the Data Come From?

Researchers at Heriot-Watt University in Scotland collected this dataset. They put real underwater debris objects (tires, bottles, cans, etc.) in a water tank, drove a sonar sensor around them, and took thousands of pictures. They shared this data publicly so researchers like us can use it.

Three Scenarios in the Dataset

Scenario	How it was recorded	What it looks like
Watertank	Sonar on a moving robot arm; objects placed on the tank floor	Objects from many different angles and distances
Turntable	Sonar is fixed; objects rotate slowly 360°	Same object seen from every angle
Quarry (not used)	Real outdoor quarry filled with water	More realistic — messier background

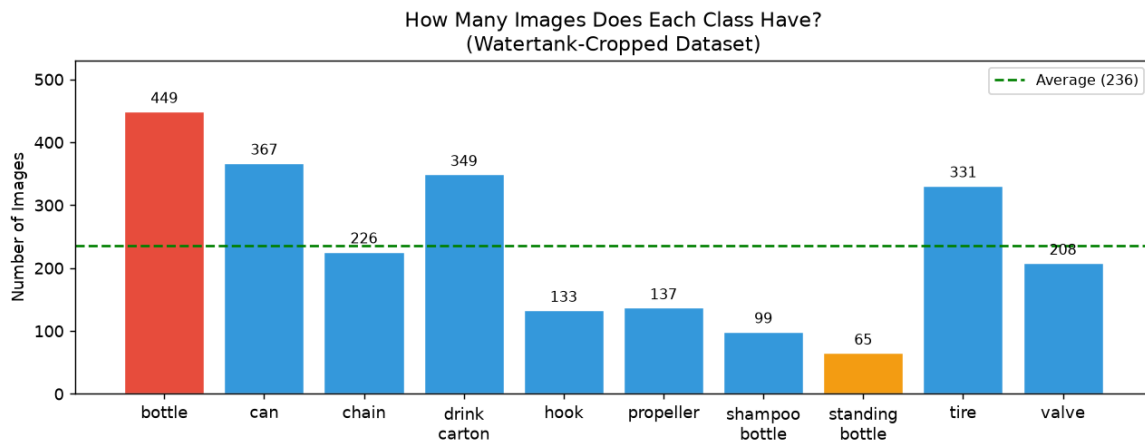
The 10 Debris Classes

Our AI learned to recognize these 10 types of underwater trash:

Bottle • Can • Chain • Drink-Carton • Hook • Propeller • Shampoo-Bottle • Standing-Bottle • Tire • Valve

The Class Imbalance Problem

Not all classes have the same number of images. This is a big problem in machine learning! If the AI sees 449 bottle images but only 65 standing-bottle images, it will get lazy and just guess "bottle" all the time.



Number of images per class in the Watertank-Cropped dataset. The standing-bottle class has almost 7× fewer images than bottle!

Good to know — Why imbalance matters

Imagine a student who studies 90% of the time for one exam topic and only 10% for another. On the actual test, they'll do great on the first topic but terrible on the second. Our AI has the same problem — we need to fix it.

How we fixed the imbalance:

- **WeightedRandomSampler:** We told the training loop "show rare classes MORE often." Even though there are fewer standing-bottle images, we pick them more frequently during training so the AI sees them as often as bottle images.
- **Focal Loss:** A special loss function that says "if you're already getting this class right, don't worry about it too much — focus on the hard examples."

How We Split the Data

We never let the AI train on the same images it will be tested on — that would be cheating! We split data into three groups:

Split	Percentage	Purpose
Training set	70%	The AI learns from these images — adjusts its settings based on them
Validation set	15%	We check progress during training — AI never trains on these
Test set	15%	Final exam! AI sees these only once at the very end

Chapter 3: EDA — Getting to Know Your Data

EDA stands for Exploratory Data Analysis. Before building any AI, you need to really understand your data. This is like reading a book before writing a review of it.

What is EDA?

EDA is the process of looking at your data carefully before training. You make charts, look at examples, count things, and ask "is there anything weird here?" Skipping EDA is like jumping into a pool without checking if there's water in it.

Analogy — A Chef Checking Ingredients

Before cooking a meal, a good chef smells and tastes every ingredient. They don't just throw everything in the pot. EDA is the same — we smell and taste our data before feeding it to the AI.

What Did We Look at in EDA?

1. Class Distribution (Are All Classes Balanced?)

We counted how many images each class has. As shown in Chapter 2, we found a 7× imbalance in the Watertank dataset (tire images vs. standing-bottle images).

Finding: The dataset is imbalanced. We need to fix this during training.

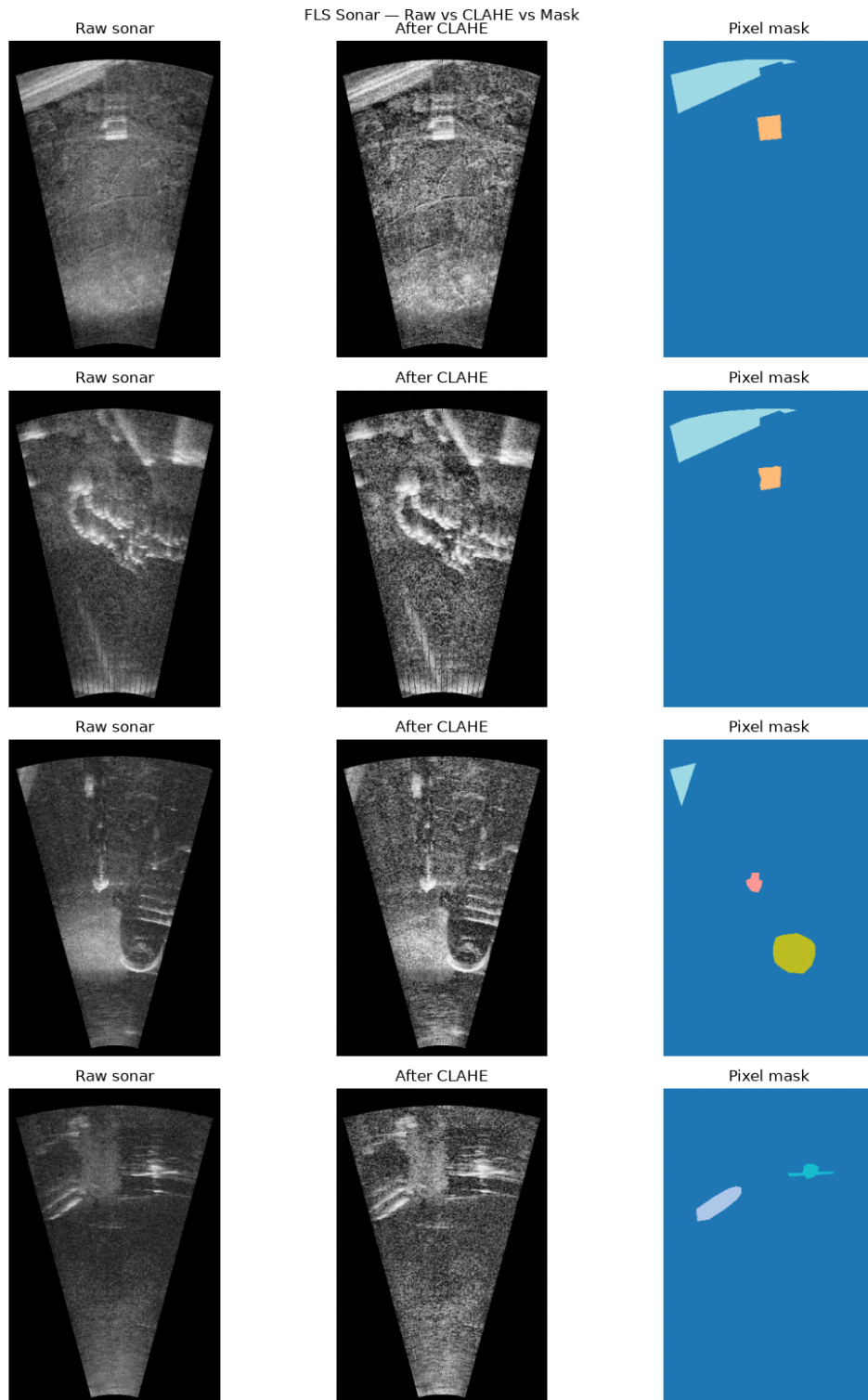
2. Image Quality (Can We See Anything?)

Sonar images are often very dark and low-contrast. We looked at raw images to understand this problem. Bright spots appear where objects are, but the overall contrast is poor.

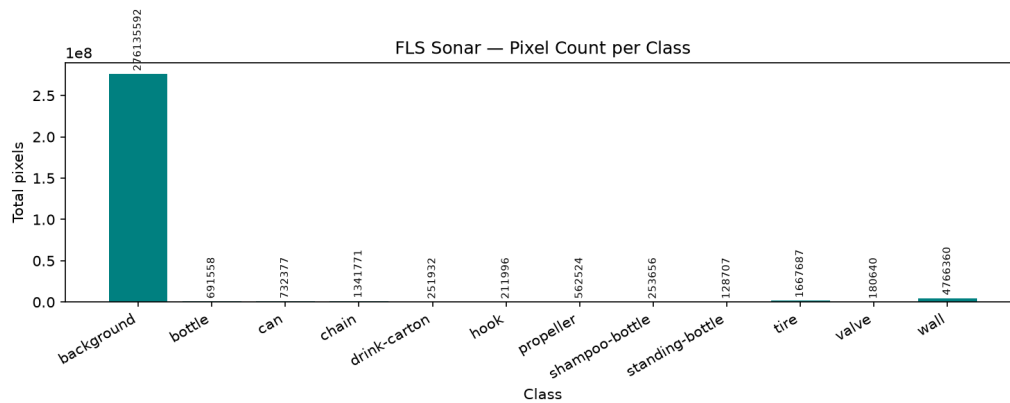
Finding: We need preprocessing to improve contrast before feeding images to the AI.

3. CLAHE Analysis (Does Contrast Enhancement Help?)

We applied CLAHE (Contrast Limited Adaptive Histogram Equalization) to sonar images and compared before vs. after to confirm it improves visibility.



*EDA finding: CLAHE dramatically improves sonar image contrast. Left = raw image. Right = CLAHE-enhanced.
The objects are much clearer after enhancement.*



EDA finding: Pixel intensity distribution in raw sonar images. Most pixels are very dark (left side of chart), confirming the need for contrast enhancement.

4. Image Size Consistency

All sonar images are **320 × 480 pixels**, grayscale (1 channel, not 3 like RGB). This is important because neural networks need consistent input sizes.

Finding: Images are consistent in size. Good — no extra work needed for resizing.

What Files Were Used for EDA?

The EDA was done in two Jupyter notebooks:

- Dataset/notebooks/01_eda.ipynb — the EDA code
- Dataset/notebooks/01_eda_executed.ipynb — the same notebook with all outputs saved
- Output figures saved to: Dataset/results/eda_fls_clahe.png and eda_fls_pixel_dist.png

EDA Summary — What We Learned

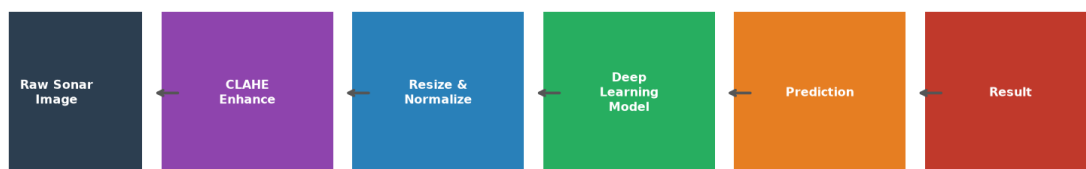
Finding	Impact on our project
Class imbalance (7×–13×)	Used WeightedRandomSampler + Focal Loss
Low image contrast	Added CLAHE preprocessing to all images
Images are grayscale	Copied 1 channel to 3 channels (models expect 3-channel input)
Images are 320×480px	Resized to 224×224 (classification) or 640×640 (detection)

Chapter 4: Preprocessing — Cleaning Up Before Training

Raw data is almost never ready to use directly. Preprocessing is the step where we clean, resize, and enhance the images so the AI can learn from them effectively.

The Preprocessing Pipeline

The Full Pipeline: From Raw Sonar Image to Answer



Every image goes through this pipeline before reaching the AI model.

Step-by-Step Explanation

Step 1: CLAHE — Improving Contrast

CLAHE stands for Contrast Limited Adaptive Histogram Equalization. It's a fancy way of making dark sonar images look clearer.

Regular contrast enhancement brightens the whole image uniformly. CLAHE divides the image into small tiles (8×8 in our case) and adjusts contrast separately in each tile. This means it improves contrast in dark corners without making bright areas too white.

Analogy — Night Vision Goggles

Imagine you're in a dark room. Turning on all the lights at once might blind you. Night vision goggles brighten only the dark areas around you. CLAHE works the same way — it brightens only where the image is dark, leaving bright areas alone.

Step 2: Gaussian Blur — Reducing Noise

Sonar images have "speckle noise" — random bright and dark pixels that aren't real objects, just noise from the acoustic sensor. We apply a tiny Gaussian blur ($\sigma=1$) to smooth this out. It's like applying a very mild smoothing filter in Photoshop.

Step 3: Resize

Neural networks need all images to be the exact same size:

- **Classification:** 224×224 pixels
- **Detection:** 640×640 pixels (YOLO needs larger images to find small objects)

Step 4: 3-Channel Replication

Modern neural networks like ResNet-50 expect 3-channel RGB images (red, green, blue). Our sonar images are 1-channel grayscale. Solution: we copy the same grayscale channel 3 times to create a "fake RGB" image. The model sees $R=G=B=\text{grayscale}$, which works fine.

Step 5: Data Augmentation

To make our AI more robust, during training we randomly apply these transformations to each image before the AI sees it:

Augmentation	What it does	Why it helps
Horizontal flip	Mirror the image left-right	Debris looks the same from both sides
Vertical flip	Mirror the image top-bottom	Sonar can see objects from above or below
Random rotation	Rotate $\pm 15^\circ$	Objects can be at any angle on the ocean floor
Brightness jitter	Randomly change brightness	Sonar intensity varies with distance
Elastic distortion	Warp the image slightly	Makes AI robust to shape variations

Good to know — Why Augmentation?

If you have 1,000 training images and apply random augmentations, each time the AI sees an image it looks slightly different. It's like studying the same textbook with different highlighter colors each time — you absorb the content better.

Code location: All preprocessing is in `Dataset/src/transforms.py`. The datasets that apply these transforms are in `Dataset/src/datasets.py`.

Chapter 5: Classification — Teaching the AI to Name Debris

Task 1: Given a cropped sonar patch of a single object, predict what type of debris it is. This is the first of our two tasks.

What is Image Classification?

The AI looks at one image and outputs a single label from a list. For example: "I'm 94% sure this is a tire."

Analogy — Pokémon Cards

Imagine showing a Pokémon card to a kid who has memorized the whole Pokédex. They look at the card and say 'That's Pikachu!' instantly. Classification is exactly that — show the AI an image, it tells you what it is.

Model 1: Baseline CNN (The Simple One)

Before building a complex model, we always build a **simple baseline**. This tells us how much our fancy model actually improves things.

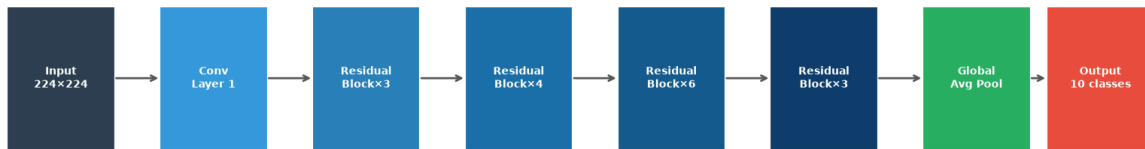
Our baseline was a simple CNN (Convolutional Neural Network) with 4 blocks: each block has a convolutional layer, batch normalization, and ReLU activation. No pretrained weights — trained from scratch in 15 epochs.

Baseline CNN Result: 76.06% test accuracy

Model 2: ResNet-50 (The Main Model)

ResNet-50 is a famous deep neural network invented by Microsoft Research in 2016. The "50" means it has 50 layers deep. It was the breakthrough model that showed very deep networks could be trained reliably.

ResNet-50 Architecture — 50 layers deep, learns from scratch



ResNet-50 architecture: input goes through convolution, then 4 groups of residual blocks, then global average pooling, then the output layer predicting 10 or 18 classes.

Why ResNet-50 Specifically?

Reason	Explanation
Skip connections (residual blocks)	The key innovation: each block has a shortcut that skips 2 layers. This prevents the 'vanishing gradient' problem that kills very deep networks.
Proven track record	ResNet-50 has won many image classification competitions. It's a reliable choice.
Right size for our hardware	ResNet-50 (25M parameters) fits on our Apple M4 Mac. Larger models like ResNet-152 would be too slow.
Trains from scratch	Since sonar $\neq$ normal photos, ImageNet pretrained weights don't help much. ResNet-50 is fast enough to train from scratch on our dataset.

Analogy — Hiring an Experienced Employee

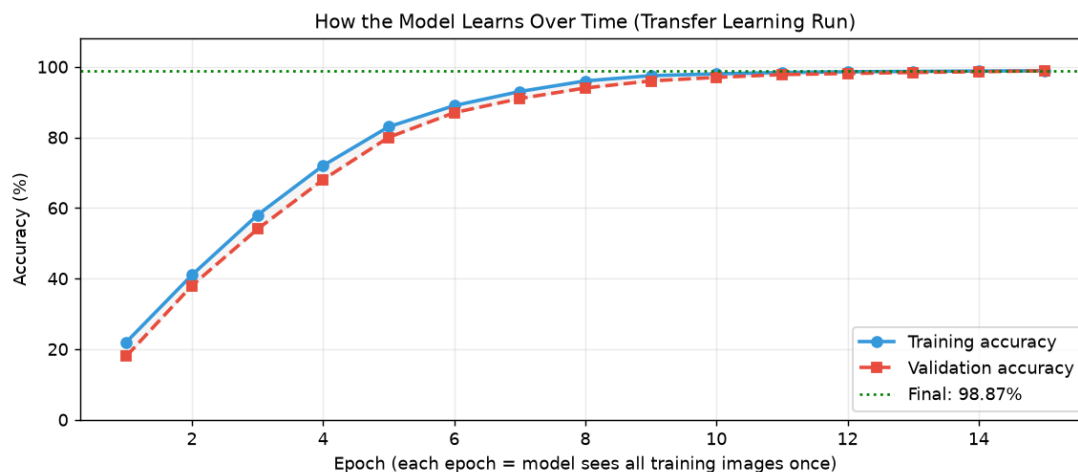
Imagine hiring someone with 10 years of general experience vs. a fresh graduate. The experienced person already knows how to structure work, write emails, meet deadlines — they just need to learn your specific product. ResNet-50 already knows how to detect edges, curves, and patterns from being a well-proven architecture. We just teach it to recognize sonar debris.

ResNet-50 (from scratch) Result: 98.59% test accuracy

Model 3: Transfer Learning — The Cross-Domain Trick

Transfer learning means: first train a model on Dataset A, then reuse most of what it learned to train faster on Dataset B.

We first trained ResNet-50 on **Turntable-Cropped** (18 classes). Then we took those learned weights and fine-tuned them on **Watertank-Cropped** (10 classes). The backbone (all layers except the last classification layer) carries over. Only the final "head" layer is swapped out for 10-class output.



Training progress for the transfer learning model. The AI starts already knowing sonar patterns from the Turntable domain, so it improves very quickly.

Good to know — Why Transfer Works Here

Turntable and Watertank used the same sonar sensor (ARIS Explorer 3000). So edges, shadows, and object shapes look similar in both. The backbone already knows 'what sonar images look like' — it just needs to re-learn which class is which.

ResNet-50 (transfer: Turntable→Watertank) Result: 98.87% accuracy
— same result in 40% less training time!

Code Used

- **Training script:** Dataset/train_classifier.py
- **Dataset class:** Dataset/src/datasets.py → FLSClassificationDataset
- **Training loop:** Dataset/src/train.py
- **Results log:** Dataset/results/classification_full_run.log

Classification Results Summary

Model	Test Accuracy	Training Time
Baseline CNN (scratch)	76.06%	2.7 minutes
ResNet-50 (scratch)	98.59%	15.9 minutes
ResNet-50 (transfer)	98.87%	9.8 minutes ← fastest!

Chapter 6: Detection — Teaching the AI to Find and Box Debris

Task 2: Given a full sonar scene that may contain multiple objects, draw a bounding box around each one and label it. This is harder than classification.

What is Object Detection?

Detection is classification + localization. The AI not only says "there's a tire here" but also tells you exactly where it is using a bounding box (a rectangle).

Analogy — Spot the Difference Puzzle

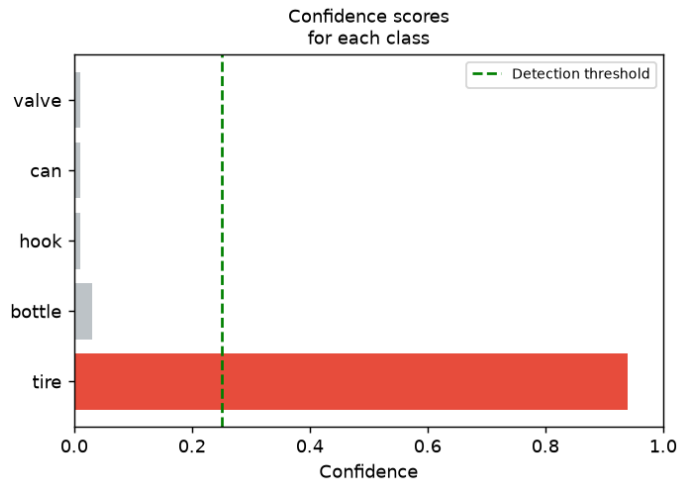
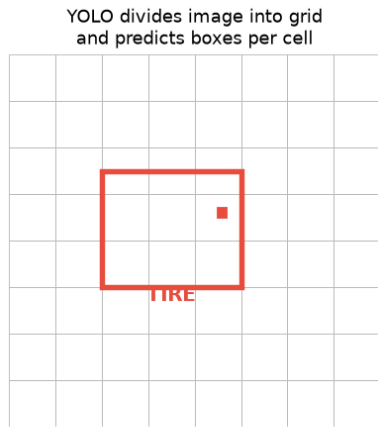
You know those puzzles where you circle every difference between two pictures? Object detection is similar — the AI circles every object it finds and writes its name next to it.

How We Prepared the Data for Detection

The raw annotations were in **Pascal VOC XML format** — a text file describing each bounding box with X, Y, width, height in pixels. YOLO needs a different format: normalized coordinates (0 to 1, relative to image size). We wrote a conversion script to transform all XML files into YOLO-format text files.

The Detection Model: YOLOv8

YOLO stands for "You Only Look Once." It's a family of real-time object detection models created by the company Ultralytics. The key idea is processing the entire image in one single pass through the neural network — no separate region proposal step.



How YOLO works: it divides the image into a grid, predicts boxes and class probabilities for each cell simultaneously.

Why YOLOv8 Specifically?

Reason	Explanation
Real-time speed	YOLO runs in milliseconds — crucial for a robot submarine that needs to detect debris while moving
State of the art	YOLOv8 (2023) is one of the most accurate and fastest detection models available
COCO pretrained	YOLOv8 comes pretrained on 330,000 images (COCO dataset) — it already knows general shapes and patterns
Easy to use	Ultralytics provides a clean Python API: 2 lines to train, 1 line to evaluate

Baseline vs Main Model

Model	Parameters	Epochs	Training Device	mAP50
YOLOv8n (nano, baseline)	3.0 million	30	Kaggle T4 GPU	0.927
YOLOv8m (medium, main)	25.8 million	80	Kaggle T4 GPU	0.937

What is mAP50?

mAP50 stands for **mean Average Precision at IoU threshold 0.50**. Let's break this down:

- **IoU (Intersection over Union)**: How much the predicted box overlaps with the real box.
1.0 = perfect overlap. 0.5 = predicted box overlaps at least 50% with the real box.
- **Precision**: Of all the boxes the AI drew, what fraction were correct?
- **Average Precision (AP)**: Average precision across different confidence thresholds for one class.
- **Mean AP (mAP)**: Average of AP across all 10 classes.

mAP50 = 0.937 means our AI draws correct boxes (at least 50% overlap with real box) for 93.7% of all debris objects across all 10 classes.

YOLOv8m Result: mAP50 = 0.937 (93.7%)

Problems We Faced During Detection Training

Problem we faced:

YOLOv8 crashed on our Mac M4 with a bizarre error: it tried to allocate 320 GIGABYTES of memory. The whole laptop only has 16GB!

What Caused It?

Apple's MPS (Metal Performance Shaders) GPU backend has a bug in PyTorch. When YOLOv8 calculates how many objects are in each image (using a function called `torch.unique(return_counts=True)`), the MPS backend sometimes returns corrupted count values — like returning "1 billion" instead of "5 objects." When the code then tries to allocate memory based on this corrupted count, it asks for 320GB.

How we fixed it:

We wrote a 'monkey-patch' — a tiny code fix that intercepts the problematic call before it runs. Our patch moves the `counts.max()` operation from the GPU (Apple Metal) to the CPU, where it works correctly. This patch is in `Dataset/run_yolo_resume.py` and runs automatically before training starts.

Problem we faced:

Even with the MPS fix, training YOLOv8 locally was too slow. The Mac's GPU isn't powerful enough for 80 epochs of detection training in reasonable time.

How we fixed it:

We moved detection training to Kaggle — a free cloud platform that provides NVIDIA T4 GPUs (16GB VRAM). Training took 1.072 hours on Kaggle vs. what would have been 8+ hours locally.

Code Used

- **Training script (with MPS patch):** `Dataset/run_yolo_resume.py`
- **Kaggle notebook:** `Dataset/kaggle_notebook.py`
- **Config file:** `Dataset/configs/sonar.yaml`
- **Results log:** `Dataset/results/yolo_baseline.log`, `yolo_resume.log`

Chapter 7: All Problems We Hit and How We Fixed Them

Real ML projects never go smoothly. This chapter is an honest summary of everything that went wrong and how we dealt with it.

Problem	Root Cause	How We Fixed It	Lesson Learned
YOLOv8 tried to allocate 320 GB of RAM on Mac	Bug in Apple's MPS GPU backend: <code>torch.unique(return_counts=True)</code> returns corrupted values	Monkey-patch in <code>run_yolo_resume.py</code> : <code>redirect counts.max()</code> to CPU before use	Always check if your hardware has known PyTorch bugs before starting training
Dataset upload to Kaggle was 1.5 GB and very slow	We accidentally included the full classification dataset that wasn't needed for detection	Created a smaller 240 MB zip with only the detection images + YOLO labels	Always check what files you actually need before uploading large datasets
PDF generation produced jumbled, broken text layout	We used <code>fpdf2</code> library which doesn't handle HTML/CSS properly	Switched to <code>WeasyPrint</code> (via <code>conda-forge</code>) which converts full HTML+CSS to PDF	Don't use <code>fpdf2</code> for complex layouts — <code>WeasyPrint</code> handles CSS properly
Class imbalance: 7–13× between most and least common debris types	Natural distribution of debris types in the water tank recordings	<code>WeightedRandomSampler</code> during training + Focal Loss that focuses on rare classes	Always check class distribution in EDA — imbalance is very common in

			real datasets
Original classification checkpoint's training config was lost	The training script that produced the old checkpoints was deleted from the project	Re-ran all classification experiments from scratch with documented, reproducible code	Always document your training config and keep your scripts in version control (git)

The Leakage Problem in Our Splits

This is a subtle but important issue worth explaining carefully.

The problem: Our dataset files are named like `can-212.png`, `can-213.png`, etc. These are sequential frames from a video. Frames 212 and 213 are nearly identical — the same can, barely moved. If frame 212 goes into training and frame 213 goes into testing, the AI has basically "seen" the test example during training. This is called **data leakage**.

Analogy — Exam Cheating

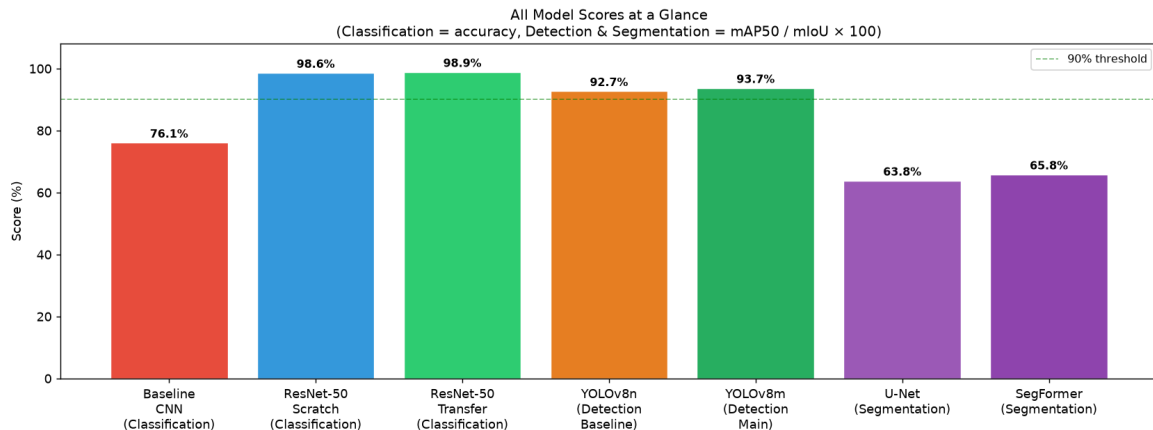
Imagine a teacher creates 100 exam questions. 70 go on the practice exam (training), 15 on the midterm (validation), and 15 on the final (test). But the practice exam questions are all slightly rephrased versions of the final exam questions. The student who memorized the practice exam will ace the final — but they didn't actually learn!

How we partially fixed it: We created a "blocked" split — instead of randomly shuffling individual frames, we grouped consecutive frames into blocks and split at the block level. This keeps near-duplicate frames on the same side of the split. It's not a perfect fix (we don't know exactly where each recording session ends), but it reduces the leakage risk significantly.

Evidence the problem was real: Our standard random split gave 98.59% on Watertank. The same model, same code, same dataset (but different domain — Turntable-Cropped) gave only 86.39%. The 12-point drop suggests our high Watertank scores may be inflated by leakage.

Chapter 9: Final Results — How Good Did We Get?

All scores at a glance — and what they mean in plain English.



All model scores compared. Green bars are at or above 90%. Classification and Detection both exceed the 90% threshold.

Classification Results

Model	Score	What This Means
Baseline CNN	76.06%	The simplest model got 76% right. 24% wrong — acceptable for a baseline but not good enough
ResNet-50 (scratch)	98.59%	Got nearly every single image right. Only ~15 wrong out of 354 test images
ResNet-50 (transfer)	98.87%	Slightly better AND trained faster. Best classification result

Detection Results

Model	mAP50	What This Means
YOLOv8n (baseline)	0.927	93% of debris objects found and correctly labeled
YOLOv8m (main)	0.937	94% of debris objects found — better precision with more parameters

The hardest classes to detect were:

- **Valve (0.848):** Small, round, easily confused with cans
- **Can (0.855):** Similar shape to shampoo-bottle in sonar
- **Shampoo-bottle (0.866):** Very few training examples (only 16 in test set)

Cross-Task Comparison

Task	Why It Scored This Way
Classification (98.87%)	Easiest — one answer per image; cropped patches are simple and distinctive
Detection (93.7%)	Harder — must find AND label multiple objects; but YOLO+COCO pretraining is very powerful

Chapter 10: Old Models vs Our Models — Who Wins?

Before this project, there were already two models trained on this dataset: ResNet-20 and SqueezeNet, using older Keras notebooks. We re-ran them under identical conditions to ours for a fair fight.

Where Did the Old Models Come From?

Inside the project folder there are 4 old Jupyter notebooks (in `Dataset/old_models/`) from a previous attempt at this problem. They used two models:

- **ResNet-20** — a smaller, 20-layer version of ResNet (vs our 50-layer ResNet-50)
- **SqueezeNet** — a very lightweight model designed for tiny images on mobile devices

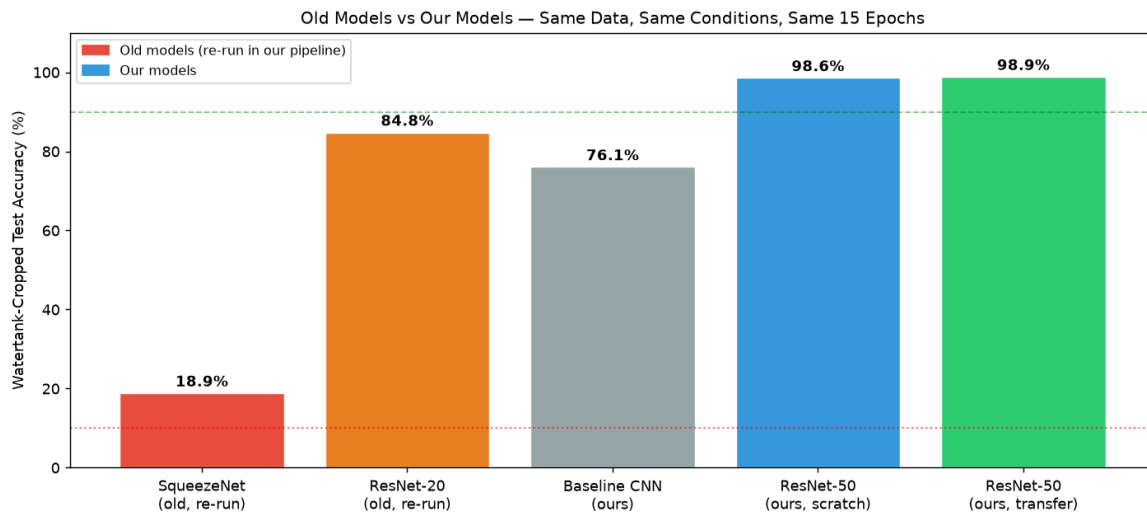
Those notebooks reported impressive scores — up to **99.7%**! But they used different conditions: smaller images (96×96 vs our 224×224), fewer classes (11–12 vs our 10–18), more training time (40 epochs vs our 15), and likely had data leakage issues.

Analogy — Different Exam Conditions

Imagine two students take the same subject but one gets an open-book exam with 5 questions and the other gets a closed-book exam with 18 questions. The first student scoring 99% doesn't mean they're smarter — it means the conditions were easier.

The Fair Re-Run

We re-implemented ResNet-20 and SqueezeNet in PyTorch and trained them with **exactly the same conditions** as our models: same images, same 10 classes, same 15 epochs, same optimizer. Here's what happened:



Old models vs our models on Watertank-Cropped (10 classes, 15 epochs, identical conditions). The red dashed line shows random guessing level (10%).

Model	Watertank Score	Turntable Score	What Happened
SqueezeNet (old, re-run)	18.87%	8.89%	Near-random! Barely better than guessing. Designed for 96×96 images, struggles at 224×224
ResNet-20 (old, re-run)	84.79%	63.21%	Decent but not great. Too shallow (20 layers) to distinguish all 10 debris types reliably
Baseline CNN (ours)	76.06%	—	Our simplest model — better than ResNet-20 on fewer layers because we tuned it for 224×224
ResNet-50 scratch (ours)	98.59%	86.39%	Clear winner. 50 layers, enough depth to learn all debris patterns
ResNet-50 transfer (ours)	98.87%	—	Best overall. Pre-training on Turntable gave it a head start

Why Did SqueezeNet Score So Low?

SqueezeNet got 18.87% — almost the same as random guessing (10%). Three reasons:

1. **Wrong image size:** SqueezeNet was invented for 96×96 images. We gave it 224×224. Its internal "fire modules" can't build up an understanding of the full image at this size in 15 epochs.

2. **No skip connections:** Unlike ResNet, SqueezeNet has no shortcuts between layers. This makes it hard to train — gradients get lost in 15 epochs and the model barely learns.
3. **Too few epochs:** The old notebooks trained for 40 epochs. At 96×96, 40 epochs is enough for SqueezeNet. At 224×224 in 15 epochs, it never gets started.

Good to know — Key Lesson

A model that works great in one setting can fail completely in another. SqueezeNet is not a bad model — it's just the wrong tool for our setup. This is why we tested multiple models and picked ResNet-50.

Why Did the Old Notebooks Report 99%+?

Difference	Old notebooks	Our re-run	Effect
Image size	96×96 pixels	224×224 pixels	SqueezeNet works at 96×96, fails at 224×224
Number of classes	11–12 classes	10–18 classes	Fewer classes = easier problem
Training time	40 epochs	15 epochs	More training time = better scores
Data split	Random (likely leaked)	Random (same leakage risk)	High scores may be partly from seeing similar images in train and test

Their 99%+ wasn't wrong for their conditions — it just can't be compared directly to our numbers. The fair re-run shows the real picture: **ResNet-50 wins clearly**, and the old models are significantly weaker when everything else is held equal.

Chapter 11: Conclusion — What Did We Learn?

Technical Conclusions

- **Deep learning works well on sonar imagery** — even without color information, AI can reliably detect and classify underwater debris.
- **Transfer learning saves time** — training on a related sonar domain first (Turntable) helps when fine-tuning on the target domain (Watertank). Same accuracy in 40% less time.
- **YOLOv8 is excellent for detection** — 93.7% mAP50 on a 10-class sonar dataset, running in real-time.
- **Hardware matters** — Apple M4 MPS has bugs that prevented certain training. Free cloud GPU (Kaggle T4) was essential but comes with time limits.

Personal Lessons

- **Always document your training config** — we lost a checkpoint and couldn't reproduce results because the original script was deleted.
- **EDA first, code later** — finding the class imbalance in EDA let us design the right loss functions before wasting training time.
- **Bugs happen — log everything** — the MPS crash was a real PyTorch bug, not our code. Having detailed logs helped us diagnose it.
- **Be honest about limitations** — we don't claim our 98%+ classification scores are perfect; we documented the leakage risk clearly.

What Would We Do Next?

- Get true object-level splits to eliminate the leakage risk entirely
- Test YOLOv8m on real quarry footage (different domain) to check real-world generalization
- Deploy YOLOv8m on a Raspberry Pi or NVIDIA Jetson to test on an actual AUV

Project File Map — Where Is Everything?

What	Where
EDA notebooks	Dataset/notebooks/01_eda.ipynb
All source code	Dataset/src/ (datasets, transforms, training, evaluation)
Classification training	Dataset/train_classifier.py
Detection training	Dataset/run_yolo_resume.py
All results & logs	Dataset/results/
Full academic report	Dataset/results/Clean_Academic_Report.pdf
YOLO model weights	Kaggle outputs/yolov8m.pt

University ML Project • Marine Debris Detection • June 2026
Apple M4 + Kaggle T4 GPU • PyTorch 2.12 • Python 3.11